A Tactic-Based Prover for Dependent Session Types: Exploiting Curry-Howard to Develop Correct-by-Construction Concurrent Processes

Kegan McIlwaine[1]
University of Wyoming, Laramie, Wyoming, USA

James Caldwell
University of Wyoming, Laramie, Wyoming, USA

Abstract

The STILL theorem prover provides an interactive, tactic-based theorem proving environment for dependent session types with polymorphism and coinduction. Session types enjoy a Curry-Howard correspondence with propositions in linear logic. STILL provides tactic engines for developing proofs in two dependent type systems: one representing the communication layer with session types, and the other representing the functional layer with a dependently typed lambda calculus. The system extracts terms from proofs in an enriched value-passing π-calculus. In linear logic, the proof rules for the so-called *multiplicative* operators require a partitioning of the linear context; this partitioning provides a significant challenge to developing proofs in a goal-directed fashion. Other systems implementing linear logic and session types require users to decide the partition at the point of tactic application or the development of a new logic and type system that allows partitioning choices to be delayed. The former solution is antithetical to goal-directed proof search and development, and the latter requires extra effort to verify consistency of the new system. The STILL system implements delayed linear context partitioning in a nondeterministic fashion without requiring modifications of the underlying proof rules. The nondeterminism inherent in partial derivations is resolved in the process of constructing tactic validations for complete derivations. To the best of our knowledge, this is a unique solution to the complexity of interactive proof systems for linear logic. We evaluate the implementation of STILL on several case studies, and report benchmarks on proof script execution time for the worst-case tensor goals (e.g., tensor sizes from 10 to 1000).

2012 ACM Subject Classification Software and its engineering → Formal methods; Theory of computation → Process calculi; Theory of computation → Linear logic

Keywords and phrases Interactive Theorem Proving, Session Types, Linear Logic

Digital Object Identifier 10.4230/LIPIcs...

Supplementary Material https://github.com/STILL-prover/STILL

Acknowledgements Generative AI tools were used in the development of the first version of the parser, REPL, and file watch systems described in this work.

1 Introduction

Since Girard's seminal publication [17], linear logic has provided a logical foundation for diverse computational systems, especially those that include constraints on resources. More than two decades later, Caires and Pfenning [8] identified a Curry-Howard correspondence between session types and intuitionistic linear logic. This was later expanded upon by Wadler [46], who established a Curry-Howard correspondence between session types and classical linear logic.

[1] Corresponding author

© Kegan McIlwaine and James Caldwell;
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

Initially introduced by Honda [24], session types provide a type discipline for Milner's π-calculus [33]. Session types allow for specifications of communication protocols. Toninho et al. [42] have developed a theory of *dependent* session types, based on intuitionistic linear logic, which extends a version of the term-passing π-calculus with polymorphism (type variables) and kinds (a type of types). While work on developing dependent session types continues [43], the work on applying these powerful theories has not gained as much traction in practice as one might have expected.

A motivation for the work described in this paper is our belief that specifications are easier to write, and more comprehensible, when they are written in richly expressive logics of the kind just mentioned. Implementations of other powerful logical systems [18, 12, 36, 27, 38, 35, 39, 37, 34] have been extraordinarily successful. We are unaware of similar systems for the theory of dependent session types.

1.1 Our Contribution

This paper describes STILL, a tactic-based theorem prover for dependent session types with coinduction and session type polymorphism. STILL stands for Session Types and Intuitionistic Linear Logic. The underlying type system of STILL is based on the dependent session type system presented in [42], with extensions that add polymorphic session types [6] and coinductive session types [41]. This theory of dependent session types intentionally leaves the functional system unspecified to maintain generality, so we have implemented the Extended Calculus of Constructions [26] as the functional system due to its simplicity and the potential it holds for future extensions.

The work presented in this paper differs from previous work in the following ways:

- To our knowledge, the STILL prover is the first standalone (*i.e.*, not embedded in another theorem prover) tactic-based prover for a dependent session type system. Previous work on dependent session types by Ciccone and Padovani [11] does not present a standalone system, but instead embeds the system within the Agda programming language.
- Primitive tactics are provided that delay context partitioning without modifying the rules of the underlying type theory. This is achieved with a nondeterministic representation of context partitions called a *subgoal candidate set*. A traversal of the tree of subgoal candidate sets is used to hide resources that have been reserved in other branches. Validations reify these trees into proofs. Previous works identify the need for delayed partitioning in both tactic-based implementations and in proof search algorithms [22, 25, 10, 45]. Unlike those works, our implementation does not modify the proof rules of the systems we base our implementation on [42, 6, 41]. We count this as a significant advantage because of the considerable effort that goes into proving soundness, progress, subject reduction, consistency, normalization, and cut elimination when modifications are made.
- The STILL prover includes a separate tactic engine and kernel for developing proofs and type checking terms in the Extended Calculus of Constructions [26] that integrates into the overall system. Other interactive systems with two layers of logic include the Abella interactive theorem prover [16], and the Hybrid tool [15] within Isabelle/HOL and Rocq. These systems have a logical layer for specification and a logical layer for reasoning about specifications. STILL differs from the other two-layer systems by providing two object-level dependent type systems inside a standalone theorem prover. The session type layer provides the ability to specify properties of protocols, and the dependently typed

functional layer provides the ability to specify properties of the values communicated by those protocols.

- Extracted proof terms are π-calculus terms rather than λ-calculus terms. Prior works that support program extraction from linear type systems either provide a type checker for the π-calculus [11, 47] or are designed to type check and extract terms of a linear λ-calculus [31, 45, 2, 28].
- The STILL prover implements theoretical extensions for coinductive and polymorphic session types within the dependent session type system developed by Toninho, Caires, and their colleagues [42, 6, 41].

This paper is organized as follows. Section 2 describes the underlying theory of dependent session types with polymorphism and coinduction that is implemented by STILL. Section 3 describes the architecture of the system, which includes the implementation of tactics with delayed context partitioning and the trusted computing base. Section 4 evaluates the usability of the system and demonstrates the application of the prover with example specifications, example extractions of π-calculus terms, and benchmarks for several types of proofs. Section 5 provides an overview of work related to the STILL prover. Section 6 concludes with discussion about the future applications of the STILL prover and the techniques that can be applied to future systems. Our declaration of generative AI use is provided in Section 7.

2 Dependent Session Types

Toninho et al. [42] have developed a theory of dependent session types where session types are used for the communication layer, and a dependent functional type system is used for the computation layer[2]. In their framework, types may be used to specify the behaviors of protocols and properties of the data being sent and received. This allows users to naturally describe the interactions between concurrent processes together with the internal computations that determine communicated values.

A sequent in this system has the form $\Psi; \Gamma; \Delta \vdash P :: z : C$. To the left of $\vdash$ are three (finite) contexts mapping variables to types. The three contexts together track typing judgments of variables and their usage restrictions. The empty context is denoted by the symbol "$\cdot$".

- Ψ is the functional context. The variables in this context have types that are used in typing functional terms in the system. Functional terms are used to define values that can be sent and received across channels. In derivations, variables in Ψ are used to prove that values transmitted in the protocol have the correct type.
- Γ is the unrestricted context. Variables in Γ have session types that correspond to exponential propositions in linear logic. These variables denote channel names with their session behaviors. These channels can be replicated zero or more times. They are used to spawn channels with fresh names in the linear context. In a way, the unrestricted context denotes shared channels acting as servers that accept zero or more connections.
- Δ is the linear context. It binds linear session behaviors to variables/channels. Each variable in the linear context must be used exactly once in a proof. The linear context denotes channels used to send and receive signals that must behave as specified by their session type.

[2] Readers interested in the full set of proof rules can find them on page 9 of [42].

$$\frac{\Psi \vdash M : \tau}{\Psi; \Gamma; \cdot \vdash [z \leftarrow M] :: z : \$\tau}\ \$\mathrm{R} \qquad \frac{\Psi, x : \tau; \Gamma; \Delta \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \$\tau \vdash P :: z : C}\ \$\mathrm{L}$$

$$\frac{\Psi, y : \tau; \Gamma; \Delta \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \exists y : \tau.A \vdash x(y).P :: z : C}\ \exists\mathrm{L} \qquad \frac{\Psi \vdash N : \tau \qquad \Psi; \Gamma; \Delta \vdash P :: z : A\{N/x\}}{\Psi; \Gamma; \Delta \vdash z\langle N\rangle.P :: z : \exists x : \tau.A}\ \exists\mathrm{R}$$

$$\frac{\Psi, x : \tau; \Gamma; \Delta \vdash P :: z : A}{\Psi; \Gamma; \Delta \vdash z(x).P :: z : \forall x : \tau.A}\ \forall\mathrm{R} \qquad \frac{\Psi \vdash N : \tau \qquad \Psi; \Gamma; \Delta, x : A\{N/y\} \vdash P :: z : C}{\Psi; \Gamma; \Delta, x : \forall y : \tau.A \vdash x\langle N\rangle.P :: z : C}\ \forall\mathrm{L}$$

Figure 1 Selected rules demonstrating dependency between the session and functional layer.

In a sequent of the form $\Psi; \Gamma; \Delta \vdash P :: z : C$, P is a term in an enriched value-passing π-calculus (see Sections 2.1 and 2.2 below). In STILL, P is extracted from a proof that dependent session type C (using channel z) is inhabited. Under the Curry-Howard interpretation, P is a proof term verifying $z : C$ under the assumptions in Ψ, Γ, and Δ. Note that variable names in Γ, Δ, and $\{z\}$ must be pairwise disjoint.

Selected rules that illustrate dependency in this system of dependent session types can be found in Figure 1. These rules demonstrate how communication of values extends the functional context, and how the protocol itself depends on the values being communicated. Sequents of the form $\Psi \vdash M : \tau$ indicate type derivations in the functional layer. Types in the term language τ are referenced in the session type level using $\$\tau$. The $\forall$ and $\exists$ rules provide quantification over values and indicate channels that receive and send terms, respectively.

2.1 Coinduction

Toninho et al. [41] extend the logical theory of intuitionistic session types with coinductive session types for corecursive processes[3]. Coinductive session types provide a system for validating protocols that run indefinitely. The rules implemented for the STILL prover (specifically νR) differ slightly from the rules described by Toninho et al. The modification allows the parameters and initial arguments to use different names, whereas the original rules require the parameters and initial arguments to have the same names.

With the addition of coinductive session types, sequents are extended with a mapping of corecursive process variables to typing contexts. As such, sequents now have the form $\Psi; \Gamma; \Delta \vdash_\eta P :: z : C$, where η is the new mapping. Type checking rules for the system of dependent session types described above are extended with η, but are otherwise left unchanged. We write $\overline{y}$ to denote lists of variables (or channels). Figure 2 presents the new rules for coinductive session types.

The first rule, νL, simply allows for unrolling a coinductive type. The next rule, νR, defines the type checking rule for corecursive process definitions. The body of the definition, P, may refer to the declared variable X. The binding information, η, tracks the context information at the point where the corecursive declaration is made. Last, the VAR rule ensures that corecursive calls are made with the same typing information after the parameter names in the contexts are renamed to the argument names.

Our modification to the rules presented in [41] is made to the νR rule by applying the renaming $\{\overline{y}/\overline{z}\}$ to the information inserted into η. This renaming allows for different parameter and argument names; otherwise, the renaming in the VAR rule would only be valid when the initial arguments and parameters have the same names.

[3] Additional proof rules for the system can be found on page 6 of [41].

$$\frac{\Psi; \Gamma; \Delta, c : A\{\nu X.A/X\} \vdash_\eta P :: d : D}{\Psi; \Gamma; \Delta, c : \nu X.A \vdash_\eta P :: d : D}\ \nu\mathrm{L}$$

$$\frac{\Psi; \Gamma; \Delta \vdash_{\eta'} P :: c : A \qquad \eta' = \eta[X(\overline{y}) \mapsto \Psi\{\overline{y}/\overline{z}\}; \Gamma\{\overline{y}/\overline{z}\}; \Delta\{\overline{y}/\overline{z}\} \vdash c\{\overline{y}/\overline{z}\} : Y]}{\Psi; \Gamma; \Delta \vdash_\eta (\texttt{corec}\ X(\overline{y}).P\{\overline{y}/\overline{z}\})\ \overline{z} :: c : \nu Y.A}\ \nu\mathrm{R}$$

$$\frac{\eta(X(\overline{y})) = \Psi; \Gamma; \Delta \vdash d : Y \qquad \rho = \{\overline{z}/\overline{y}\}}{\rho(\Psi); \rho(\Gamma); \rho(\Delta) \vdash_\eta X(\overline{z}) :: \rho(d) : Y}\ \mathrm{VAR}$$

Figure 2 Type checking rules for coinductive session types

$$\frac{\Omega, X; \Psi; \Gamma; \Delta \vdash_\eta P :: z : A}{\Omega; \Psi; \Gamma; \Delta \vdash_\eta z(X).P :: z : \forall X.A}\ \forall 2\mathrm{R}$$

$$\frac{\Omega \vdash_\eta B\ \texttt{stype} \qquad \Omega; \Psi; \Gamma; \Delta, x : A\{B/X\} \vdash_\eta P :: z : C}{\Omega; \Psi; \Gamma; \Delta, x : \forall X.A \vdash_\eta x\langle B\rangle.P :: z : C}\ \forall 2\mathrm{L}$$

$$\frac{\Omega \vdash_\eta B\ \texttt{stype} \qquad \Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : A\{B/X\}}{\Omega; \Psi; \Gamma; \Delta \vdash_\eta z\langle B\rangle.P :: z : \exists X.A}\ \exists 2\mathrm{R}$$

$$\frac{\Omega, X; \Psi; \Gamma; \Delta, x : A \vdash_\eta P :: z : C}{\Omega; \Psi; \Gamma; \Delta, x : \exists X.A \vdash_\eta z(X).P :: z : C}\ \exists 2\mathrm{L}$$

Figure 3 Type checking rules for polymorphic session types

Coinductive session types are essential for a usable session type system. In session type systems without coinduction, it is still possible for long running protocols to be type checked using the rules for replication [8]; but it is not possible to type check a long running protocol that changes over time and calls itself. Many of the processes that require coinduction to be well-typed are simple concurrent processes described in Milner's book on the π-calculus [33], *e.g.*, bit counters and semaphores.

2.2 Polymorphism

As defined at the beginning of Section 2 through Section 2.1, polymorphism exists in the functional layer of the system, but the session type layer does not allow for polymorphism. In another piece of work, Caires et al. [6] use second-order intuitionistic linear logic to define polymorphic session types. Based on this work, we have implemented the rules in Figure 3 in STILL (omitting the straightforward well-formedness rules for conciseness)[4]. These rules also include the syntactic extensions to the π-calculus that allow processes to send and receive session types. Following along with the work by Caires et al., a new context, Ω, is added for tracking type variables and checking well-formed session types. Sequents are extended with the new context, and are written $\Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : C$. Type checking rules for the system described so far are also extended with the new context, but are otherwise left unchanged.

As demonstrated by Toninho and Yoshida [44], these rules for the quantification over session types make it possible to express inductive and coinductive session types. An

[4] The full set of proof rules for this system can be found on page 7 of [6].

```
n        ::= (natural numbers)
x,y,z    ::= (variable names)
xs,ys,zs ::= x,xs | ϵ

-- Functional terms
M,N,S,T  ::= Pi x : M . N | Sigma x : M . N | lambda x : M . N
         | (M,N) : Sigma x : S . T | fst M | snd M | M N | Prop | Type n | x

-- Session types
A,B      ::= 1 | A -o B | A * B | A & B | A + B | nu x . A | !A | x
         | $M | forall x : M . A | exists x : M . A
         | forall x : stype . A | exists x : stype . A

-- Process terms
P,Q      ::= P|Q | (nu x:A) P | x[y].P | x[M].P | x[A].P | x.inl;P | x.inr;P
         | x.case(P, Q) | x(y).P | !x(y).P
         | x <-> y | [x <- M] | corec x(ys) P (zs) | x(ys)
```

Figure 4 Concrete syntax for terms and types in the STILL prover.

inductive session type $\mu X.F(X)$ can be encoded as $\forall X.!(F(X) \multimap X)$, and a coinductive type $\nu X.F(X)$ can be encoded as $\exists X.!(X \multimap F(X)) \otimes X$. For example, the session type for natural numbers would be $\forall X.!((1 \oplus X \multimap X) \multimap X)$. Example proofs in the supplemental material demonstrate validating and extracting π-calculus terms for inductive session types using polymorphism.

2.3 Putting It All Together

The STILL theorem prover uses the system described in this section with sequents of the form $\Omega; \Psi; \Gamma; \Delta \vdash_\eta P :: z : C$. Ω tracks session type variables that can be used in session types in Γ, Δ, η, and C. Ψ, the functional context, contains type judgments for variables that can be used in terms in the functional layer. Γ, the unrestricted context, contains channels with replicating session protocols. Δ, the linear context, tracks linear channels. η, the binding map, tracks the binding information for corecursive processes. To the right of the turnstile is the process being type checked, P, followed by the channel implementing the session behavior in the process, z. The rightmost element of the sequent, C, is the session type indicating the protocol of the channel. Throughout the rest of the paper, the process term may be omitted for readability. Such sequents will have the form: $\Omega; \Psi; \Gamma; \Delta \vdash_\eta z : C$.

The concrete syntax for functional terms, session types, and processes used in the STILL theorem prover are presented in Figure 4. The syntax and semantics for the session type terms and process terms are described in detail in [8, 42, 6, 41] — soundness and type preservation are also proved. The syntax, semantics, and computational properties of the functional terms are from Luo's Extended Calculus of Constructions [26].

3 Tactics for Session Types

Tactics were initially described by Gordon et al. [19] as pure functions with the following types:

```
tactic = goal -> ([goal], validation)
validation = [theorem] -> theorem
```

As Gordon et al. state, primitive tactics can be written for logical systems by inverting the inference rules, *i.e.*, reading them bottom-up. A *valid tactic* [19] has the property that if the list of theorems given to the validation function proves the list of goals returned as the result of the tactic, then the theorem output from the validation function will prove the goal originally given to the tactic. Now consider the rule for the tensor operator in the system described in Section 2:

$$\frac{\Omega; \Psi; \Gamma; \Delta_1 \vdash_\eta y : A \qquad \Omega; \Psi; \Gamma; \Delta_2 \vdash_\eta z : B}{\Omega; \Psi; \Gamma; \Delta_1, \Delta_2 \vdash_\eta z : A \otimes B} \otimes\mathrm{R}$$

Applying the inference rule requires two proofs with disjoint linear assumptions Δ_1 and Δ_2. A naive approach to writing a valid tactic for the rule requires the user to provide a partition of the assumptions when the tactic is applied. The tactic would then output the two subgoals with those user-provided partitions as the respective assumptions in each subgoal. However, this requires the user to have knowledge of exactly how the proof will partition the assumptions. Such a system would require the user to have a clear plan for resource usage in at least one branch. Otherwise, the user may pick the wrong partition, making it impossible to complete the proofs of the subgoals.

3.1 Delaying Partition Choices

In order to delay the partitioning decision, other automated and interactive systems often modify the rules of the logic. Modifications include adding constraints that must be solved to give the desired partition [45], having the sequent indicate the used and unused assumptions in the context that can be used in other branches [10, 23, 1, 47], or creating backchaining rules that delete resources in the linear context [22]. These modifications still result in proof rules that contain unknown partition results at the top of the inference rules. Constraint solving is deferred, and the leftover resource information is not known until the full proof branch is complete.

To demonstrate the issue, consider a modification to the tensor right rule based on the resource management calculus presented by Cervesato et al. [10]:

$$\frac{\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta' \vdash_\eta y : A \qquad \Omega; \Psi; \Gamma; \Delta' \backslash \Delta^O \vdash_\eta z : B}{\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta^O \vdash_\eta z : A \otimes B} \otimes\mathrm{R}$$

In this system, the context $\Delta^I \backslash \Delta^O$ indicates that the linear resources in Δ^I are available to prove the goal, and the resources that were not used in the proof are in Δ^O. While this more closely resembles the desired behavior of delaying the context partition for as long as possible, implementing a tactic that delays the partitioning is still not straightforward. A simple tactic that follows the rule from the bottom up creates two subgoals of the form $\Omega; \Psi; \Gamma; \Delta^I \backslash \Delta' \vdash_\eta y : A$ and $\Omega; \Psi; \Gamma; \Delta' \backslash \Delta^O \vdash_\eta z : B$, and the user would be expected to provide Δ' and Δ^O just as they would be expected to provide Δ_1 and Δ_2 for the tactic based on the tensor operator in intuitionistic linear logic. Modifying the notation in this way does not delay the context partition choice.

STILL tactics take as input and output a nondeterministic representation of subgoals. We call this structure, which encodes all possible partitions of the linear context, a *subgoal candidate set*. Order of the subgoals is not significant. Conceptually[5], STILL tactics have the following type:

[5] In the implementation of STILL, the state monad is used to represent proof states which allows for an imperative style of sharing which is crucial for constructing and evaluating validations.

$$
\begin{array}{l}
[\{\cdot;\cdot;\cdot;a:B,b:A \vdash_\eta z:B\otimes A\}] \\
\xrightarrow{\texttt{TensorR}} \left[\left\{\begin{array}{l} \cdot;\cdot;\cdot;a:B,b:A\vdash_\eta y:B, \\ \cdot;\cdot;\cdot;a:B\vdash_\eta y:B, \\ \cdot;\cdot;\cdot;b:A\vdash_\eta y:B, \\ \cdot;\cdot;\cdot;\cdot\vdash_\eta y:B, \end{array}\right\}, \left\{\begin{array}{l} \cdot;\cdot;\cdot;\cdot\vdash_\eta z:A, \\ \cdot;\cdot;\cdot;b:A\vdash_\eta z:A, \\ \cdot;\cdot;\cdot;a:B\vdash_\eta z:A, \\ \cdot;\cdot;\cdot;a:B,b:A\vdash_\eta z:A \end{array}\right\}\right] \\
\xrightarrow{\texttt{Id a}} [\{\cdot;\cdot;\cdot;b:A\vdash_\eta z:A\}] \xrightarrow{\texttt{Id b}} []
\end{array}
$$

Figure 5 Example of all possible partitions in proofs applying the `TensorR` tactic.

```
subgoalCandidates = [goal]
tactic = subgoalCandidates -> ([subgoalCandidates], validation)
validation = [theorem] -> theorem
```

As tactics are applied, invalid subgoal candidates are pruned from the sets. If the collection of subgoal candidate sets is empty, that shows that the derivation tree is complete — or — there is no well-formed derivation tree resulting from the tactics as applied. Which of these two holds can tentatively be identified if the last rule applied was an axiom rule. But whether the derivation is a valid proof can only be determined by running the validation.

Figure 5 demonstrates an example of proof development beginning with the singleton subgoal candidate set $[\{\cdot;\cdot;\cdot;a:B,b:A\vdash_\eta z:B\otimes A\}]$. Arrows indicate a transition between subgoal candidate sets using the tactic in the label. In the example, applying the `TensorR` tactic to the initial subgoal candidate set results in the transition to two subgoal candidate sets, with each subgoal in a set representing a context partition choice for a subgoal. For readability of the figure, the individual subgoal candidates that allow the completion of the proof are colored blue.

Applying the `Id a` tactic reserves `a`, and eliminates subgoal candidates where the tactic application is invalid (*i.e.*, `a` is not the only element in the linear context or the type of `a` does not match the type of the active channel). There is only a single subgoal candidate at this point. Applying the `Id b` tactic completes the proof. Following this simple proof demonstrates how the partition of the linear context is determined as the proof is constructed. The choices are not baked into the rules, and a partitioning choice is not explicitly made by the user.

3.2 Representing Subgoal Candidates

Treating tactics nondeterministically avoids the need to modify the system of dependent session types at the core of the STILL theorem prover. However, enumerating and tracking all subgoal candidates individually requires storing 2^n subgoals in each candidate set for every application of the `TensorR` tactic, where n is the number of elements in the linear context. This quickly becomes infeasible, even for modestly sized proofs.

The subgoal candidates in a set only differ in the choice of partitions for the linear context. For that reason, the STILL prover represents subgoal candidate sets deterministically as a single sequent, except for the linear context, which is stored as all currently unclaimed resources. For example, the `TensorR` tactic for the tensor right rule outputs the subgoal candidates for the left and right operand with the full linear context available on both sides. Section 3.3 and Section 3.4 describe how linear resources are reserved in each branch of the proof.

3.3 Applying Tactics

We now move from theoretical foundations to the concrete implementation. The STILL prover is implemented in Haskell and uses the state monad to implement tactics in a comparable manner to the techniques described by Martin et al. [30], and by Martin and Gibbons [29]. The proof state holds information about tactic history, variable usage, and open goals (subgoal candidate sets). The user can then interact with the system by selecting the current candidate set, applying tactics to the selected candidate set, or viewing other state information.

The proof state of the STILL prover contains a map of names to subgoal candidate sets, together with a list of the names of the open candidate sets. Every candidate set, except the initial one, is the result of a tactic application from a previous subgoal. The subgoal candidate set information also contains reserved resource names, a reference to the previous subgoal candidate set, references to the next subgoal candidate sets, and the branch type of the applied tactic if the subgoal candidate has been expanded.

When a tactic is applied to a subgoal candidate set, it updates the candidate set information with any resources it reserved in the linear context, and creates the next subgoal candidate sets and the justification that will run when the proof is complete. Tactics create subgoal candidate sets according to their corresponding inference rule as described in Section 3.2. Those new subgoal candidate sets are then added to the candidate set map in the state and the list of open candidate sets. Subgoal candidate set references to their parent and children subgoal candidate sets are also created by the application of a tactic. This creates a tree structure from the initial goal to the current open candidate sets.

The corresponding inference rule for a tactic that creates multiple candidate sets may require the linear context for the proofs of those goals to be disjoint or equal. This branch type distinction corresponds to inference rules for multiplicative and additive connectives, respectively. When a tactic is applied to a candidate set, the candidate set is marked as expanded with the corresponding branch type of the tactic (or as a trunk expansion if less than two subgoal candidate sets are created).

3.4 Tracking Resources in the Subgoal Candidate Tree

Due to tactics outputting sequents with all unreserved linear resources as the representation of subgoal candidate sets, all resources would appear available in the linear context when the user views the candidate sets on opposite sides of a multiplicative branch. This introduces a new problem for the users; they might use resources that have already been used elsewhere in the proof. If the two candidate sets that use the same resource are on opposite sides of the proof of a multiplicative operator, the full proof will fail. This problem is solved in the STILL theorem prover by tracking the reserved resources in the candidate set tree.

When the user views the sequent representing a candidate set, the system traverses the subgoal candidate set tree and collects all resource names that are not available to that candidate set. Those resources are then hidden and treated as unavailable. The recursive traversal that collects the resources to hide has the following steps:

1. Take a subgoal candidate set as input.
2. Initialize an empty set of resource names, R.
3. For the current subgoal candidate set, look up the parent subgoal candidate set.
4. If the previous subgoal candidate set was expanded by a multiplicative tactic then get all reserved resource names in branches other than the current branch (*i.e.*, the parent's other children) and add them to R.

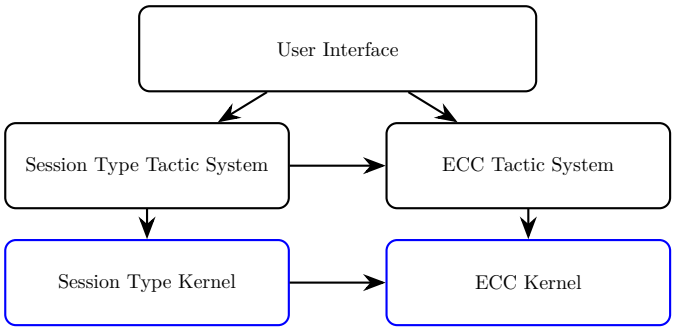

Figure 6 The architecture of the STILL prover.

5. Recursively apply this algorithm with the parent subgoal candidate set as input and union the result with R.
6. Return R.

By applying this algorithm and hiding resources that are returned, the resources that have been used on the opposite sides of a multiplicative branch are automatically tracked. When a resource is in the set returned by the algorithm for a given candidate set input, that indicates there is a multiplicative inference rule (*e.g.*, the rule for tensor on the right) in the path from the root goal to the subgoal candidate set in the proof derivation that will have that resource allocated to the other branch. Preventing the use of those resources in other branches prevents the construction of proofs with invalid partitions.

3.5 The Trusted Computing Base

The STILL theorem prover is organized into separate modules for the type system kernels and tactic implementations. The architecture diagram of the STILL prover can be seen in Figure 6 where nodes represent system components (typically a single Haskell module). Arrows between nodes indicate the source node references the target node. Nodes colored in blue represent the trusted computing base of the system. These are the two kernels for proof checking: one for the system of dependent session types described in Section 2, and one for the Extended Calculus of Constructions (ECC) [26]. These kernels check the correctness of proof objects in the two systems. The dependent session type kernel invokes the ECC kernel when it checks an inference rule that contains a functional proof object. Assuming the correctness of the implementation of the kernels, acceptance of a proof object corresponds to a valid derivation of the concluding sequent (including the π/λ-calculus term extraction).

Both kernels contain a Haskell datatype definition for a proof object whose constructors correspond to attempted inference rule applications for the system, and a validation function that determines whether the proof object should be accepted or not, *i.e.*, each inference rule is applied correctly. This includes well-formedness checking of polymorphic session types and checking corecursive calls match the recorded typing environment after substitution. The validation functions in both kernels rely on algorithms for capture-avoiding substitution, α-renaming, and fresh name generation. The ECC kernel also relies on a normalization algorithm for determining convertibility and universe cumulativity between two terms.

The dependent session type kernel also checks the multiplicative rule applications for disjoint linear contexts between the premises. Proof objects produced by the tactic system that attempt to use non-disjoint partitions will be caught when the proof object is checked.

```
*?c>> ; ; ; a:B, b:A |- z:B ⊗ A
π> apply TensorR
Tensor right side tactic applied.
 ?d>> ; ; ; a:B, b:A |- c:B
*?e>> ; ; ; a:B, b:A |- z:A
π> apply IdA
IdA tactic applied.
*?d>> ; ; ; a:B |- c:B
π>
```

```
*?c>> ; ; ; a:A, b:A |- z:A ⊗ A
π> apply TensorR
Tensor right side tactic applied.
 ?d>> ; ; ; a:A, b:A |- c:A
*?e>> ; ; ; a:A, b:A |- z:A
π> apply IdA
IdA tactic applied.
*?d>> ; ; ; b:A |- c:A
π>
```

Figure 7 Example prompts and commands.

We note that under the correct implementation of the tactic system, the user should not be able to construct these invalid proofs interactively. The traversal algorithm and resource hiding prevent the use of resources in both premises of a multiplicative inference rule. However, the kernel still checks for disjoint linear contexts, and rejects proof objects with invalid multiplicative rule applications.

While we attempt to implement the tactic systems correctly, they may be unsound. For that reason, the kernels reside in modules that do not reference the tactics or prover state for either system. Trust in the STILL prover does not rely on the soundness of the tactic implementations, the subgoal candidate tree, the subgoal candidate sets, or resource hiding algorithms. Only kernel validation grants theorem status.

4 Evaluating the System

This section evaluates the usefulness of the STILL prover with the resource hiding algorithm and two tactic engines. There are three modes in which the system can run: the script mode that runs proof scripts, the file watch mode that reruns the proof script as the file is updated, and the REPL mode for experimentation. The script running mode can optionally return benchmarks for the scripts that are discussed in Section 4.3. The resource hiding algorithm is used in the file watch and REPL modes as the user develops proofs.

4.1 Usefulness of Resource Hiding

The examples in Figure 7 show two REPL sessions. One session has two different types of resources (`b:A` and `a:B`), and the other has multiple of the same type of resources (`A`). These examples demonstrate how both subgoal candidate sets receive the unclaimed resources in the linear context after the `TensorR` tactic is applied. A context partitioning has not been determined at that point.

After the `TensorR` tactic creates multiplicative branches in the candidate set tree, resource hiding is demonstrated with the application of the `IdA`[6] tactic to reserve a resource on one branch of the candidate set tree (contexts are printed as $\Omega; \Psi; \Gamma; \Delta$ and left blank if empty). Before `IdA` is applied, both `a` and `b` are available in the linear context of the sequents representing both open subgoal candidate sets. After the tactic is applied, only one resource is available in the open candidate set, because the other is reserved on the opposite side of a multiplicative branch in the subgoal tree.

[6] The automatic version of the `Id` tactic. Automatic versions of tactics that use resources in the contexts are provided in addition to versions that allow specifying which resource to use.

The example on the left follows the proof development in Figure 5. The example on the right is provided as a demonstration of a proof where both branches have the ability to use the same resources in the development of their proofs. The STILL tactic engine ensures the user does not have to track which resource of type `A` has been used to prove subgoal `?e`. This example generalizes to a larger proof with more complex steps, during which the user would quickly lose track of which resources have been used in either branch without the hiding algorithm.

4.2 The Functional Subsystem

Developing proofs in the STILL prover requires interacting with the functional subsystem. When the user applies a tactic for session types that require discharging a functional typing subgoal (*e.g.,* the tactic for $R in Figure 1), the tactic is suspended pending the completion of the proof of the initialized functional subgoal. When a tactic is applied to the functional subgoal, the system checks if the proof is complete, and if so, the original session type tactic is resumed, expanding candidate sets and reserving resources.

Users see subgoals for an in-progress functional proof associated with a session type subgoal with names written as `<sessionSubgoal>.<functionalSubgoal>`. Tactics are applied to functional subgoals using the `apply` command in the same way they are applied to session type subgoals. Tactic expressions in the session type and functional systems also have the same tactical operators. In this way, STILL provides a seamless transition between the two tactic engines during proof development.

4.3 Example Proofs and Benchmarks

The STILL source code in the supplementary material comes with example proofs that demonstrate its features. The proof scripts can be executed by running `./still Proofs/*.still`, and the benchmark table for the proof scripts can be generated with the benchmark argument `./still benchmark Proofs/*.still`. Our benchmark results from running the examples[7] are presented in Table 1. The time column represents an average time over five executions of the script. A single execution includes the time to read the file from the disk, parse the file, execute the commands, and print the final message to the command line. The final message is printed to the command line to ensure full evaluation of the proof script due to the lazy nature of Haskell; otherwise, the function that returns the final proof state may not be fully executed and the benchmarks would not correctly evaluate the system. The examples in the table have been chosen to exercise the system features together, to stress test the traversal algorithm, and to assess the faithfulness of the implementation of the theory presented by Toninho, Caires, and their colleagues [42, 6, 41].

In Table 1 the Bank, CloudServer, and Counter modules assess the faithfulness of the implementation of the theory of dependent session types with polymorphism and coinduction. The Bank module validates a bank and ATM client protocol from [42]. The CloudServer module validates a polymorphic cloud server protocol, and a file conversion instance of the protocol from [6]. The Counter module validates a coinductive protocol for a bit counter which includes processes that store bits and are spawned as more storage is required from [41].

[7] Benchmarks were taken on a 2019 MacBook Pro with a 2.4 GHz 8-Core Intel Core i9 processor and 32 GB 2667 MHz DDR4 memory.

Module	Theorems	Total Subgoals Total Inference Rules	Max Subgoals Max Inference Rules	Time (s)
Auction	3	104 189	38 115	0.009079
Bank	2	51 675	30 497	0.006347
CloudServer	2	25 25	14 14	0.000794
Counter	3	72 198	32 128	0.003192
LargeMul	1	3997 2999	3997 2999	3.536581

Table 1 Benchmarks for the STILL prover.

The LargeMul module stress tests the resource hiding traversal algorithm in the worst-case scenario: traversing the entire subgoal tree for every tactic application. It contains an expression with 1000 variables combined using the tensor operator. Implication of the same expression, but reversed, allows the proposition to be proved, *i.e.*, `(a * ... * l38)` implies `(l38 * ... * a)`. This module demonstrates that the traversal algorithm for reserving and hiding resources does not impede proof checking and development in a meaningful way. The proof of the proposition also demonstrates the power of the subgoal candidate sets. A single tactic expression `ImpliesR; TensorLA+; (TensorR; (IdA <|> Skip))+` proves the entire proposition.

Additional experiments testing how the performance scales can be found in the supplementary material. Tensor expressions like those in the LargeMul module are evaluated with sizes from 10 to 1000, and the results show quadratic scaling. These are worst-case benchmarks as the tensor operator expands subgoals with multiplicative branches in the subgoal tree, so every subgoal must be traversed to hide resources for each tactic application. Further work is required to see if performance can be improved.

4.4 The Auction Specification

Figure 8 presents the English auction protocol from the Auctions module (omitting the proof for space), and demonstrates the power of dependent session types alongside coinductive and polymorphic session types. The auction protocol begins with the seller providing the type of the item being auctioned. The item type has type `stype`, so it represents a service being offered (including ones that may transfer a tangible object). Then the protocol introduces types for bids and a bid validation and a currency type family. The bid validation type family represents a validation parameterized by the old bid and the current bid. A term of the type `validNewB b1 b2` is a proof that `b2` is a valid new bid when checked against the previous bid `b1`. The currency dependent type family represents the currency being provided, and is parameterized by the amount of the currency. With this, the auction is polymorphic with respect to the item being offered, the validation of the bids, and the type of the currency. Finally, before bidding begins, the item is sent to the auction process, and the seller provides a channel that will receive the winning bid.

After the auction begins, the first bid amount can be provided along with a validation of the first bid against the zero bid. After that, the correct amount of currency must be provided. The current highest bidder also provides a channel that can either receive the item in the case that they win, or their money back in the case that their bid is beaten. After the

```
theorem english:
  "forall ItemType:stype. forall bid:Type 1. forall zeroB:bid.
    forall validNewB:(Pi b1:bid. Pi b2:bid. Type 1).
      forall currency:(Pi b:bid.Type 1).ItemType -o
        (forall b:bid. $currency b -o 1) -o forall firstB:bid.
          forall firstValid:validNewB zeroB firstB. $currency firstB -o
            (ItemType + $currency firstB -o 1) -o
              nu Y. (forall newB:bid.forall nextValid:validNewB firstB newB.
                $currency newB -o (ItemType + $currency newB -o 1) -o Y) & 1"

z(ItemType).z(bid).z(zeroB).z(validNewB).z(currency).z(a).z(c).
  z(firstB).z(firstValid).z(d).z(e).
    (corec AUC(item, ownerC, curB, winnerC, oldB, lastB, curValid).z.case(
      inl: z(newB).z(nextValid).z(h).z(i).((ν j) winnerC[j].
        (j.inr;[curB <-> j] | AUC(item, ownerC, h, i, lastB, newB, nextValid))),
      inr: ownerC[lastB].((ν f) ownerC[f].
        ([curB <-> f] | (ν g) winnerC[g].(g.inl;[item <-> g] | 0)))))
    (a, c, d, e, zeroB, firstB, firstValid)
```

Figure 8 English auction specification and extracted π-calculus term.

$$
\begin{aligned}
\mathit{postOffice} &\triangleq \forall \mathit{itemObjectType} : \mathit{Type}\ 1.\forall \mathit{item} : \mathit{itemObjectType}.\$\mathit{delivery}\ \mathit{itemObjectType}\ \mathit{item} \\
\mathit{bookSeller} &\triangleq \exists \mathit{startPrice} : \mathit{int}.\exists \mathit{bidValid} : (\Pi b1 : \mathit{int}.\Pi b2 : \mathit{int}.\mathit{Type}\ 1). \\
&\qquad \exists \mathit{bookForSale} : \mathit{book}.(\forall b : \mathit{int}.\$\mathit{usd}\ b \multimap 1) \\
\mathit{bookBuyer} &\triangleq \forall \mathit{validator} : (\Pi b1 : \mathit{int}.\Pi b2 : \mathit{int}.\mathit{Type}\ 1).\exists \mathit{buyerAmount} : \mathit{int}.\forall \mathit{curPrice} : \mathit{int}. \\
&\qquad \exists \mathit{buyerValid} : \mathit{validator}\ \mathit{curPrice}\ \mathit{buyerAmount}.\$(\mathit{usd}\ \mathit{buyerAmount}) \otimes \\
&\qquad\qquad (\forall \mathit{bookBought} : \mathit{book}.(\$\mathit{usd}\ \mathit{buyerAmount} \oplus \$\mathit{delivery}\ \mathit{book}\ \mathit{bookBought}) \multimap 1)
\end{aligned}
$$

$$\cdot; \mathit{book} : \mathit{Type}\ 1, \ldots, \mathit{usd} : \mathit{Type}\ 1; \cdot; \mathit{bookSeller}, \mathit{bookBuyer}, \mathit{auction}, \mathit{postOffice} \vdash_{\cdot} z : 1$$

Figure 9 Book Auction Protocol

first bid is fully processed, the protocol enters a coinductive session that repeats the process of receiving a bid, or terminates once a termination signal is given.

The linearity of the session types ensures that once another bid is received, the money is returned to the previous highest bidder. The dependent session types ensure that bidders only receive their money back, because the refund channel is typed with the current bid amount. No other amount of currency can be sent back to the bidder. Additionally, the seller can only receive the highest bid at the end of the auction, because the coinductive session type requires that channels that exist at the start of the auction remain available as the auction continues (modulo renaming according to the VAR rule). Finally, when the auction receives the signal to terminate (*i.e.*, enters the right side of the & connective), the item must be sent to the highest bidder, because that is the only channel that remains that can receive the item, and the seller receives the money from the highest bid, because that is the only channel that remains that can receive money.

The π-calculus process extracted from the proof of the validity of the auction protocol can also be seen in Figure 8. It has been formatted for readability and to save space. It enjoys the properties explained above that have been guaranteed by the dependent session types: bids must be validated, losers receive their money back, the winner receives the item, and the seller of the item receives the money from the winning bid.

Additionally, a verification that a book auction protocol satisfies the specification of the English auction is included in the Auction module. It validates the protocol in Figure 9. In the book auction, the post office creates deliveries for items, the book seller offers a book for sale with a bid validator, and the book buyer provides a valid bid with the expectation of the delivery of the book if they win. The types for the book, bids, currency, and delivery are assumed in the functional context. The validation of the protocol corresponds to the creation of a process that enforces the correct interaction between the other four processes in order to complete the auction. This validation demonstrates the use of coinductive, polymorphic, and dependent session types together. Even though the book seller only offers a book for sale, the post office process can be used to create the delivery that the book buyer expects. Finally, the coinductive auction protocol on the left of the turnstile must terminate (according to the rule for 1 on the right). The auction terminating ensures that the items and money are transferred correctly.

5 Related Work

Implementations of dependent session types are most closely related to the work presented in this paper. In particular, the work by Ciccone and Padovani [11] presents a dependently typed π-calculus in Agda. Their system also includes a layer for a functional type system that is separate from the session type layer in the same way the system presented by Toninho et al. [42] does. However, it does not include the multiplicative operators, so context partitioning is not addressed in their work. Other implementations of dependent session types can be found in the RAST [14] and Granule [28] programming languages. RAST provides type level dependency with arithmetic refinements, and Granule provides type level dependency on natural number inputs.

Due to the Curry-Howard correspondence identified between linear logic and session types, previous work on interactive theorem proving for linear logic is also closely related to the work presented in this paper. A system for interactive development of intuitionistic linear logic has been implemented in Isabelle by Kalvala and De Paiva [25], and a deep embedding of intuitionistic linear logic in Isabelle has been developed by Smola and Fleuriot [40]. Standalone provers have also been implemented for linear logic. These systems include the Janet theorem prover developed by Trifunovski [45], and the Click and Collect theorem prover by Callies and Laurent [9]. All of these systems have modified the inference rules in the system to allow for delayed context partitioning, except for the Click and Collect prover which requires the partition to be provided at the moment a multiplicative tactic is applied.

Also tangentially related to the STILL theorem prover is Idris 2 [5], which is built on Quantitative Type Theory [31, 2]. While it is not directly related to session types or linear logic, Idris 2 presents a dependently typed language and proof environment that must track resources as they are consumed. However, Quantitative Type Theory is not a theory of session types (there is no context partitioning for the $\otimes$ rule), and Idris 2 is not a tactic-based theorem prover, so the relation between STILL and Idris 2 is marginal.

Proof search systems for linear logic and type checking implementations of session types are also related to our work on delayed context partitioning. Proof search systems include the work by Hodas and Miller [22] and Cervesato et al. [10] on efficient resource management for the Lolli prover. The work of Harland et al. [21] on the Lygon theorem prover also identifies the problem of finding the correct context partition for the multiplicative rules. Allais [1] presents a type checker for a linear λ-calculus using leftovers. Zalakain and Dardha [47] use the same techniques to implement a type checker for a π-calculus. The resource management

calculus provided by Cervesato et al. [10], and the leftover type system presented by Allais [1] both result in systems that return the resources that were not consumed in one branch of the premises of a multiplicative rule. These are systems that behave similar to the way the STILL prover hides resources; but the delayed partitioning is encoded in the calculus/type system, while the STILL prover delays the choice at the tactic level and validates the resulting partitions at the kernel level.

6 Conclusions and Future Work

To our knowledge, the STILL prover is the first standalone tactic-based interactive theorem prover for dependent session types. The system provides a concrete implementation of the theoretical system presented by Toninho, Caires, and their colleagues over multiple works [42, 6, 41], but there are similar theoretical systems. Griffith and Gunter [20] and Ciccone and Padovani [11] have both presented dependently typed π-calculi with a separate dependently typed functional layer. The system developed by Ciccone and Padovani is embedded in Agda. Additionally, the Session Abstract Machine (SAM) [7] provides a type checker and interpreter for a process calculus using classical linear logic. The SAM also contains a simple functional layer with operations on integers, Booleans, and strings.

The STILL prover provides π-calculus process extractions from completed proofs. Additional work is required to translate these π-calculus processes with value-passing, type-passing, and corecursion into forms that can be run on existing systems that use process calculi. There are at least two approaches: to extract π-calculus terms from STILL proofs in a form that can be evaluated, or to translate the π-calculus terms used by STILL into an existing executable form. Target systems are SAM processes, ProVerif processes [4], or blockchain smart contract languages based on process calculi [13, 3, 32]. Having an executable form of STILL extracts would allow for simulation, further verification, and deployment of the extracted processes. For example, a translation of the English auction process in Figure 8 into a smart contract would provide the protocol's guarantees of liquidity, refunding losing bids, and asset transfer. Formal verification of the translations would be required to ensure the preservation of the invariants given by the session type of the process.

Previous implementations of theorem provers for linear logic and session types have managed the context partitioning in one of two ways. The first is to require the user to provide the partition when refining a multiplicative goal, and continuing the proof using that choice [9]. The second is to modify the rules of the logic or type system to allow the choice of the partition to be delayed [10, 1, 47]. The STILL prover implements a third technique that models all partition choices nondeterministically using subgoal candidate sets. Users of the STILL theorem prover never explicitly provide the partition. The partitions are represented using subgoal candidate sets as the proof is constructed, and finally checked at completion. With this technique, a new session type system did not need to be re-developed and re-verified. We expect the same tactic implementation techniques can be used to implement other flavors of session types and linear logic in their own tactic-based theorem provers without changing the underlying proof rules.

7 Declaration of Generative AI Use

The code for the system described in this paper was implemented without the use of generative AI except in the following areas: (i) parsing the terms into Haskell datatypes and (ii) creating a generic script interpreter, file watch, and REPL system.

References

1 Guillaume Allais. Typing with leftovers - a mechanization of intuitionistic multiplicative-additive linear logic. In Andreas Abel, Fredrik Nordvall Forsberg, and Ambrus Kaposi, editors, *23rd International Conference on Types for Proofs and Programs (TYPES 2017)*, volume 104 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 1:1–1:22, Dagstuhl, Germany, 2019. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2017.1, doi:10.4230/LIPIcs.TYPES.2017.1.

2 Robert Atkey. Syntax and semantics of quantitative type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, LICS '18, page 56–65, New York, NY, USA, July 2018. Association for Computing Machinery. URL: https://dl.acm.org/doi/10.1145/3209108.3209189, doi:10.1145/3209108.3209189.

3 Massimo Bartoletti, Riccardo Marchesin, and Roberto Zunino. Secure compilation of rich smart contracts on poor utxo blockchains. In *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*, pages 235–267, 2024. doi:10.1109/EuroSP60621.2024.00021.

4 Bruno Blanchet, Martín Abadi, and Cédric Fournet. Automated verification of selected equivalences for security protocols. *The Journal of Logic and Algebraic Programming*, 75(1):3–51, February 2008. doi:10.1016/j.jlap.2007.06.002.

5 Edwin Brady. Idris 2: Quantitative Type Theory in Practice. In Anders Møller and Manu Sridharan, editors, *35th European Conference on Object-Oriented Programming (ECOOP 2021)*, volume 194 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:26, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2021.9, doi:10.4230/LIPIcs.ECOOP.2021.9.

6 Luís Caires, Jorge A. Pérez, Frank Pfenning, and Bernardo Toninho. Behavioral polymorphism and parametricity in session-based communication. In Matthias Felleisen and Philippa Gardner, editors, *Programming Languages and Systems*, pages 330–349, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

7 Luís Caires and Bernardo Toninho. The session abstract machine. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 206–235, Cham, 2024. Springer Nature Switzerland.

8 Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In Paul Gastin and François Laroussinie, editors, *CONCUR 2010 - Concurrency Theory*, page 222–236, Berlin, Heidelberg, 2010. Springer. doi:10.1007/978-3-642-15375-4_16.

9 Etienne Callies and Olivier Laurent. Click and coLLecT An Interactive Linear Logic Prover. In *5th International Workshop on Trends in Linear Logic and Applications (TLLA 2021)*, Rome (virtual), Italy, June 2021. URL: https://hal-lirmm.ccsd.cnrs.fr/lirmm-03271501.

10 Iliano Cervesato, Joshua S. Hodas, and Frank Pfenning. Efficient resource management for linear logic proof search. In Roy Dyckhoff, Heinrich Herre, and Peter Schroeder-Heister, editors, *Extensions of Logic Programming*, pages 67–81, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

11 Luca Ciccone and Luca Padovani. A dependently typed linear π-calculus in agda. In *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming*, PPDP '20, New York, NY, USA, 2020. Association for Computing Machinery. doi:10.1145/3414080.3414109.

12 R. L. Constable, S. F. Allen, H. M. Bromley, W. R. Cleaveland, J. F. Cremer, R. W. Harper, D. J. Howe, T. B. Knoblock, N. P. Mendler, P. Panangaden, J. T. Sasaki, and S. F. Smith. *Implementing mathematics with the Nuprl proof development system*. Prentice-Hall, Inc., USA, 1986.

13 Ankush Das, Stephanie Balzer, Jan Hoffmann, Frank Pfenning, and Ishani Santurkar. Resource-aware session types for digital contracts. In *2021 IEEE 34th Computer Security Foundations Symposium (CSF)*, pages 1–16, 2021. doi:10.1109/CSF51468.2021.00004.

14 Ankush Das and Frank Pfenning. Rast: A language for resource-aware session types. *Logical Methods in Computer Science*, Volume 18, Issue 1:7024, January 2022. doi:10.46298/lmcs-18(1:9)2022.

15 Amy Felty and Alberto Momigliano. Hybrid. *Journal of Automated Reasoning*, 48(1):43–105, January 2012. doi:10.1007/s10817-010-9194-x.

16 Andrew Gacek. The abella interactive theorem prover (system description). In Alessandro Armando, Peter Baumgartner, and Gilles Dowek, editors, *Automated Reasoning*, pages 154–161, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

17 Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, January 1987. doi:10.1016/0304-3975(87)90045-4.

18 Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. *Edinburgh LCF*, volume 78 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 1979. URL: http://link.springer.com/10.1007/3-540-09724-4, doi:10.1007/3-540-09724-4.

19 Michael J. Gordon, Arthur J. Milner, and Christopher P. Wadsworth. *Edinburgh LCF*, volume 78 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 1979. URL: http://link.springer.com/10.1007/3-540-09724-4, doi:10.1007/3-540-09724-4.

20 Dennis Griffith and Elsa L. Gunter. Liquidpi: Inferrable dependent session types. In Guillaume Brat, Neha Rungta, and Arnaud Venet, editors, *NASA Formal Methods*, pages 185–197, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

21 James Harland, David Pym, and Michael Winikoff. Programming in lygon: An overview. In Martin Wirsing and Maurice Nivat, editors, *Algebraic Methodology and Software Technology*, pages 391–405, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

22 J. S. Hodas and D. Miller. Logic programming in a fragment of intuitionistic linear logic. *Information and Computation*, 110(2):327–365, May 1994. doi:10.1006/inco.1994.1036.

23 Joshua S. Hodas, Pablo López, Jeffrey Polakow, Lubomira Stoilova, and Ernesto Pimentel. A tag-frame system of resource management for proof search in linear-logic programming. In Julian Bradfield, editor, *Computer Science Logic*, pages 167–182, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.

24 Kohei Honda. Types for dyadic interaction. In Eike Best, editor, *CONCUR'93*, page 509–523, Berlin, Heidelberg, 1993. Springer. doi:10.1007/3-540-57208-2_35.

25 Sara Kalvala and Valeria De Paiva. Mechanizing linear logic in isabelle. In *In 10th International Congress of Logic, Philosophy and Methodology of Science*, volume 24, 1995.

26 Zhaohui Luo. *Extended calculus of constructions*. PhD thesis, The University of Edinburgh, 1990. Accepted: 2016-01-19T16:28:16Z publisher: The University of Edinburgh. URL: https://era.ed.ac.uk/handle/1842/12487.

27 Zhaohui Luo and Robert Pollack. *LEGO proof development system: User's manual*. LFCS, Department of Computer Science, University of Edinburgh, 1992.

28 Danielle Marshall and Dominic Orchard. Non-linear communication via graded modal session types. *Information and Computation*, 301:105234, 2024. URL: https://www.sciencedirect.com/science/article/pii/S0890540124000993, doi:10.1016/j.ic.2024.105234.

29 Andrew Martin and Jeremy Gibbons. A monadic interpretation of tactics. Accessed from https://www.cs.ox.ac.uk/jeremy.gibbons/publications/tactics.pdf, 2002.

30 Andrew Martin, Ray Nickson, and Mark Utting. A tactic language for ergo. In *Formal Methods Pacific '97*, 1997.

31 Conor McBride. *I Got Plenty o' Nuttin'*, pages 207–233. Springer International Publishing, Cham, 2016. doi:10.1007/978-3-319-30936-1_12.

32 Kegan McIlwaine and James Caldwell. Verifying Smart Contract Transformations Using Bisimulations. In Diego Marmsoler and Meng Xu, editors, *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *Open Access Series in Informatics (OASIcs)*, pages 9:1–9:19, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.FMBC.2025.9, doi:10.4230/OASIcs.FMBC.2025.9.

33 Robin Milner. *Communicating and mobile systems: the π-calculus*. Cambridge University Press, USA, 1999.

34 Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, page 625–635, Berlin, Heidelberg, 2021. Springer-Verlag. URL: https://doi.org/10.1007/978-3-030-79876-5_37, doi:10.1007/978-3-030-79876-5_37.

35 Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002. doi:10.1007/3-540-45949-9.

36 Bengt Nordström, Kent Petersson, and Jan M Smith. *Programming in Martin-Löf's type theory*, volume 200. Oxford University Press Oxford, 1990.

37 Ulf Norell. *Towards a practical programming language based on dependent type theory*. PhD thesis, Chalmers University of Technology, Göteborg, Sweden,, 2007. URL: https://www.cse.chalmers.se/~ulfn/papers/thesis.pdf.

38 Lawrence C. Paulson. Isabelle: The next 700 theorem provers. *CoRR*, cs.LO/9301106, 1993. URL: https://arxiv.org/abs/cs/9301106.

39 The Rocq Development Team. *The Rocq Prover*, September 2025. doi:10.5281/zenodo.17473943.

40 Filip Smola and Jacques D. Fleuriot. Linear resources in isabelle/hol. *J. Autom. Reason.*, 68(2), May 2024. doi:10.1007/s10817-024-09698-2.

41 Bernardo Toninho, Luis Caires, and Frank Pfenning. Corecursion and non-divergence in session-typed processes. In Matteo Maffei and Emilio Tuosto, editors, *Trustworthy Global Computing*, page 159–175, Berlin, Heidelberg, 2014. Springer. doi:10.1007/978-3-662-45917-1_11.

42 Bernardo Toninho, Luís Caires, and Frank Pfenning. Dependent session types via intuitionistic linear type theory. In *Proceedings of the 13th international ACM SIGPLAN symposium on Principles and practices of declarative programming*, page 161–172, Odense Denmark, July 2011. ACM. URL: https://dl.acm.org/doi/10.1145/2003476.2003499, doi:10.1145/2003476.2003499.

43 Bernardo Toninho, Luís Caires, and Frank Pfenning. A decade of dependent session types. In *Proceedings of the 23rd International Symposium on Principles and Practice of Declarative Programming*, PPDP '21, page 1–3, New York, NY, USA, October 2021. Association for Computing Machinery. URL: https://dl.acm.org/doi/10.1145/3479394.3479398, doi:10.1145/3479394.3479398.

44 Bernardo Toninho and Nobuko Yoshida. *Polymorphic Session Processes as Morphisms*, pages 101–117. Springer International Publishing, Cham, 2019. doi:10.1007/978-3-030-31175-9_7.

45 Maksim Trifunovski. *An Interactive Proof Assistant for Linear Logic*. Bachelor of arts with departmental honors, Wesleyan University, Middletown, CT, 2017. URL: https://digitalcollections.wesleyan.edu/object/ir-682, doi:10.14418/wes01.1.1339.

46 Philip Wadler. Propositions as sessions. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*, ICFP '12, page 273–286, New York, NY, USA, 2012. Association for Computing Machinery. doi:10.1145/2364527.2364568.

47 Uma Zalakain and Ornela Dardha. π with leftovers: A mechanisation in agda. In *Formal Techniques for Distributed Objects, Components, and Systems: 41st IFIP WG 6.1 International Conference, FORTE 2021, Held as Part of the 16th International Federated Conference on Distributed Computing Techniques, DisCoTec 2021, Valletta, Malta, June 14–18, 2021, Proceedings*, page 157–174, Berlin, Heidelberg, June 2021. Springer-Verlag. URL: https://doi.org/10.1007/978-3-030-78089-0_9, doi:10.1007/978-3-030-78089-0_9.